

Lernziele:

- Zuweisungen
- Verständnis für die Unterscheidung von links und rechts assoziative Operatoren.
- Wie in jedem Entwicklungszyklus:
 - Code Refactoring
 - Inkrementelle Softwareentwicklung und Vertiefung des bisher Gelernten
 - Umgang mit Fehlermeldungen!!

Zielsetzung

Der Compiler soll jetzt Zuweisungen (Assignments) wie zum Beispiel

```
x = 3 * 4
```

unterstützen. Außerdem soll ein Programm aus mehr als nur einer Expression bestehen können. Kommt nach einer Expression ein Strichpunkt, dann soll dies ein Expression-Statement darstellen, und ein Programm soll aus einer beliebig langen Sequenz von Expression-Statements bestehen können. Der Exit-Code des Programms soll dann der Wert des letzten Expression-Statements sein. Beispielsweise sollte das Programm

```
x = 3;
y = 4;
y = x + y;
x = x * 2;
x * y;
```

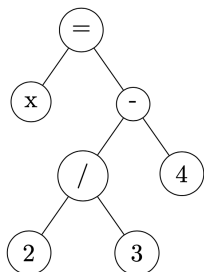
den Exit-Code 42 liefern.

Schritt 1 (Erweiterung der Expression-Trees)

Ein guter Anfang für diese umfangreiche Erweiterung sind die Expression-Trees. Bei einer Zuweisung wie

```
x = 2/3 - 4;
```

soll beim Parsen folgender Expression-Tree erzeugt werden:



Aber bis dahin ist es ein langer Weg, denn wir müssen fast überall im Compiler etwas anpassen:

- Der Lexer muss das Zeichen '=' erkennen.
- Die Grammatik für den Parser muss erweitert werden.
- Die in `simple.isa` beschriebene Hardware unterstützt nur ALU-Instruktionen und muss um Befehle für den Speicherzugriff (Store und Fetch) erweitert werden.
- Der Code-Generator muss erweitert werden, so dass man damit Fetch- und Store-Instruktionen generieren kann.

Diese Liste ist deprimierend lang, und das macht die Expression-Trees zu einem geeigneten Kandidaten für den ersten Schritt, bei dem es das Ziel ist, dass man ein Bildchen wie oben erzeugen kann. Wenn man dann tatsächlich so ein Bildchen erzeugt hat, ist das ein Erfolgserlebnis, das einem Auftrieb, Energie und Schwung für den Rest gibt.

Die Vorgehensweise ist dabei die übliche: Erst mal ein Testprogramm schreiben, in dem man auch festlegt, wie die Funktionen zur Erzeugung von so einem Expression-Tree benutzt werden können.

Zum Beispiel:

- Analog zu `createIntegerExpr` soll es eine Funktion `createIdentifierExpr` geben, um einen Knoten für eine Variable zu erzeugen.
- Eine Zuweisung ist einfach eine Binary-Expression, bei der der Operator eben eine Zuweisung anstatt einer Addition oder Multiplikation ist.

Ich habe dazu mein Testprogramm wie folgt angepasst:

```

1 @ <stdio.h>
2 @ "expr.hdr"
3 @ "gen.hdr"
4
5 fn main()
6 {
7     local expr: -> Expr;
8
9     expr = createBinaryExpr(EXPR_SUB,
10                             createBinaryExpr(EXPR_DIV,
11                                                 createIntegerExpr((string)"2"),
12                                                 createIntegerExpr((string)"3")),
13                             createIntegerExpr(string"4"));
14     expr = createBinaryExpr(EXPR_ASSIGN,
15                             createIdentifierExpr((string)"x"),
16                             expr);
17     printExprTree(expr);
18     printf("\n");
19
20     // genReleaseReg(loadExpr(expr));
21
22     releaseExpr(expr);
23 }

```

Die Zeilen 9 bis 13 waren bereits so im alten Testprogramm und stellen einen Baum für den Ausdruck $2 / 3 - 4$ dar. Die Zeilen 14 bis 16 habe ich neu hinzugefügt. Hier wird ein Zuweisungsknoten (Assignment-Node) erzeugt. Der linke Child-Node ist ein Knoten für die Variable `x`, der rechte ist einfach der zuvor erzeugte Expression-Tree. Zeile 20 habe ich auskommentiert, da hier Assembler-Code erzeugt wird und ich mich damit erst beschäftigen möchte, wenn das obige Bildchen erzeugt wird.

Jetzt soll das Programm schrittweise funktionstüchtig gemacht werden:

1. Wenn man jetzt `make` ausführt (macht das!), wird man vom Compiler natürlich eine Fehlermeldung bekommen, denn `EXPR_ASSIGN` ist unbekannt. Kein Problem, in `expr.hdr` muss man für `ExprKind` eine Enum-Konstante hinzufügen. Da ein Assignment-Node zwei Child-Nodes hat, ist auch klar, wo man die Konstante hinzufügen muss.
2. Wenn man jetzt nochmals `make` ausführt (macht das!), bekommt man eine neue Fehlermeldung, denn `createIdentifierExpr` ist unbekannt. Kein Problem, in `expr.hdr` kopiert man die Deklaration von `createIntegerExpr` und benennt die Kopie um in `createIdentifierExpr`, denn beide kann man ja analog benutzen und haben deshalb das gleiche Interface.
3. Wenn man jetzt nochmals `make` ausführt (macht das!), bekommt man einen Fehler vom Linker, da die Funktion `createIdentifierExpr` zwar deklariert wurde (deshalb hat der Compiler nicht gemeckert), aber nirgends definiert wurde. Deshalb findet der Linker das Symbol natürlich in keinem der Object-Files. Kein Problem, in `expr.abc` kopiert man die Definition der Funktion `createIntegerExpr` und benennt in der Kopie die Funktion um in `createIdentifierExpr`. Im Funktionskörper ändert man:

```

n->kind = EXPR_INTEGER;
n->decimal = val;

```

um in

```

n->kind = EXPR_IDENTIFIER;
n->identifier = val;

```

Den Rest lässt man so, wie er ist.

4. Wenn man jetzt nochmals `make` ausführt (macht das!), bekommt man einen Fehler vom Compiler, dass es kein Member `identifier` gibt. Kein Problem, in `expr.hdr` kopiert man in der Deklaration von Struct `Expr` die Zeile für die Deklaration des Members `decimal` und ändert in der Kopie den Namen des Members zu `identifier`. Jetzt sollte zumindest alles wieder übersetzen!

5. Wenn man `xtest_expr` jetzt ausführt, bekommt man ein **Assertion-Failure**. In der Fehlermeldung sieht man, dass dies in `expr.abc` ausgelöst wurde und bekommt auch eine Zeilennummer. Damit stellt man schnell fest, dass in dem Switch-Statement eigentlich alle Arten von Binary-Expressions behandelt werden sollten, aber der Fall `EXPR_ASSIGN` wird bis jetzt noch nicht behandelt. Kein Problem, man muss nur für den Fall `EXPR_ASSIGN` die Ausgabe eines Gleichheitszeichens analog zu den anderen Fällen hinzufügen. Hier war mein größtes Problem, dass LaTeX im Expression-Baum das Gleichheitszeichen später nicht angezeigt hat. Nach einer Google-Recherche habe ich herausgefunden, dass man nicht `=` sondern `{=}` erzeugen muss. Letztlich habe ich also den Fall so hinzugefügt:

```
case EXPR_ASSIGN: printf("{=}"); break;
```

6. Wenn man `xtest_expr` ausführt, gibt es keinen Laufzeitfehler mehr. Das Problem ist jetzt "nur", dass die Ausgabe nicht stimmt, das Programm also nicht macht, was es soll. Es gibt jetzt:

```
[{=}[ ][-[/[2][3]] [4]]
```

aus. Das Problem ist das leere Klammerpaar `[]` nach `{=}`. Da sollte eigentlich `x` drin stehen. Aber kein Problem, das Problem ist, dass in meinem Code noch davon ausgegangen wird, dass eine Primary-Expression immer ein Integer-Knoten ist. Deshalb steht in `printExprTree` dieses Fragment:

```
if (expr->kind >= EXPR_PRIMARY && expr->kind < EXPR_PRIMARY_END) {
    printf(" [ %s ]", expr->decimal);
} else if (expr->kind >= EXPR_BINARY && expr->kind < EXPR_BINARY_END) {
```

Da sollte man jetzt natürlich auch eine Fallunterscheidung einbauen wie diese:

```
switch (expr->kind) {
    case EXPR_INTEGER: printf(" [ %s ]", expr->decimal); break;
    case EXPR_IDENTIFIER: printf(" [ %s ]", expr->identifier); break;
    default: assert(0);
}
```

Mit dieser Anpassung sollte die Ausgabe jetzt so aussehen:

```
[{=}[x][-[/[2][3]] [4]]
```

Um den LaTeX-Code für die Expression-Trees zu erzeugen, braucht ihr von Session 13 die Datei `tree_header.tex`

```
\documentclass[preview, margin=0.2cm]{standalone}
\usepackage{forest}
\begin{document}
\begin{forest}
```

und `tree_footer.tex`

```
\end{forest}
\end{document}
```

Dann kann man mit:

```
(cat tree_header.tex; ./xtest_expr ; cat tree_footer.tex) > tree.tex
```

die LaTeX-Datei `tree.tex` erzeugen und mit:

```
lualatex tree.tex
```

die PDF-Datei `tree.pdf` erzeugen. Das Bild wird noch nicht so hübsch aussehen wie das obere, und darum kümmern wir uns jetzt im nächsten Schritt.

Schritt 2

Wir gehen die Sache gleich richtig an, sodass man auch mehr als einen Expression-Tree mit LaTeX visualisieren kann. Dazu wird es eine neue Funktion `printExprLatexTree` geben, die in `expr.hdr` (als `extern`) deklariert und in `expr.abc` implementiert wird. Aber zunächst passen wir das Testprogramm an, damit klar ist, wie man die Funktion benutzen kann:

```

1 @ <stdio.h>
2 @ "expr.hdr"
3 @ "gen.hdr"
4
5 fn main()
6 {
7     local expr: -> Expr;
8
9     expr = createBinaryExpr(EXPR_SUB,
10                             createBinaryExpr(EXPR_DIV,
11                                             createIntegerExpr((string)"2"),
12                                             createIntegerExpr((string)"3")),
13                             createIntegerExpr(string"4"));
14     printExprLatexTree(expr);
15     expr = createBinaryExpr(EXPR_ASSIGN,
16                             createIdentifierExpr((string)"x"),
17                             expr);
18     printExprLatexTree(expr);
19     printf("\n");
20
21     // genReleaseReg(loadExpr(expr));
22
23     releaseExpr(expr);
24 }
```

Die Funktion wird in Zeile 14 und Zeile 18 aufgerufen. Und wie man sieht, kann man die Funktion genau wie `printExprTree` benutzen. Wenn wir mit dem Schritt fertig sind, wird das Programm folgenden LaTeX-Code erzeugen (ich habe so lange daran gearbeitet, bis ich am Ende mit dem Ergebnis zufrieden war, also fragt mich nicht nach dem Wieso und dem Weshalb):

```

\item
\l[
\begin{forest}
for tree={draw,circle,calign=fixed edge angles}
[ - [ / [ 2 ] [ 3 ] ] [ 4 ] ]
\end{forest}
\l]
\item
\l[
\begin{forest}
for tree={draw,circle,calign=fixed edge angles}
[ {=} [ x ] [ - [ / [ 2 ] [ 3 ] ] [ 4 ] ] ]
\end{forest}
\l]
```

Die Expression-Trees sind innerhalb einer Math-Umgebung eingebettet und kommen nach einem LaTeX-Item. Den LaTeX-Header `tree_header.tex` habe ich geändert zu:

```

\documentclass[preview, margin=0.2cm]{standalone}
\usepackage{forest}
\begin{document}
\begin{enumerate}
```

und `tree_footer.tex` zu

```

\end{enumerate}
\end{document}
```

Bei der Implementierung von `printExprLatexTree` kann man natürlich die Funktion `printExprTree` aufrufen und davor und danach den zusätzlichen LaTeX-Code ausgeben. Hier ist das

kniffligste die Ausgabe eines Backslashes. Um '\\' auszugeben, muss man im Formatstring '\\'

```
fn printExprLatexTree(expr: ->Expr)
{
    printf("\\item\n");
    printf("\\[\n");
    printf("\\begin{forest}\n");
    printf("for tree={draw,circle,calign=fixed edge angles}\n");
    printExprTree(expr);
    printf("\n");
    printf("\\end{forest}\n");
    printf("\\]\n");
}
```

verwenden: }

Aufgabe: Setzt das Ganze um und testet, ob ihr aus der Ausgabe ein PDF erzeugen könnt.

Schritt 3 (Lexer erweitern)

Wenn ihr `xtest_lexer` startet, seht ihr, dass aktuell die Zeichen `=` und `;` nicht erkannt werden:

```
MCL:s19 lehn$ ./xtest_lexer
;=
1.1: BAD_TOKEN (;)
1.2: BAD_TOKEN (=)
```

Meine Idee bei der Erweiterung des Lexers ist, dass diese Zeichen analog zum Zeichen `+` (ich hätte aber auch `-`, `*` oder `/` rauspicken können) erkannt und verarbeitet werden sollen. Die Enum-Konstante für `+` ist `PLUS`. Also suche ich in `lexer.hdr` und `lexer.abc` nach Zeilen, in denen `PLUS` vorkommt, kopiere sie und passe sie an:

1. In `lexer.hdr` kommt `PLUS` nur einmal vor, und zwar in der Deklaration von `TokenKind`. Also kopiere ich die Zeile zweimal und ändere die Konstante jeweils in `SEMICOLON` und `EQUAL`.
2. In `lexer.abc` kommt `PLUS` in zwei verschiedenen Zeilen vor. Hier muss man etwas mehr mitdenken, was und wieviel man kopiert und dupliziert, aber man muss nicht sehr viel denken ;-)

Aufgabe

Passt den Lexer so an, dass die zusätzlichen Zeichen erkannt werden und `xtest_lexer` folgende Ausgabe beim Testen erzeugt:

```
MCL:s19 lehn$ ./xtest_lexer
;=
1.1: SEMICOLON (;)
1.2: EQUAL (=)
```

Schritt 4 (Grammatik und damit den Parser erweitern)

In diesem Schritt wird die Grammatik zunächst nur so erweitert, dass ein Programm aus einer Sequenz von Expression-Statements besteht. Die Expressions bleiben aber zunächst unverändert, es werden also noch keine Zuweisungen unterstützt.

```
prog = { statement }
statement = expr ";"
expr = term { ( "+" | "-" ) term }
term = factor { ( "*" | "/" ) factor }
factor = decimal-literal
        | "(" expr ")"
```

Neu sind also nur die Produktionsregeln `prog` und `statement`. Das richtige Vorgehen wäre jetzt, dafür zwei Parse-Funktionen nach dem üblichen Schema zu implementieren. Aber ich bin da etwas ungeduldig und baue das, was neu ist, direkt in das Test-Programm `xtest_parser.abc` ein. Der Grund ist, dass ich eigentlich nur die Zuweisungen unterstützen möchte, die machen aber

nur dann Sinn, wenn man mehr als eine Expression im Programm haben kann. Die saubere Unterstützung der neuen Produktionsregeln baue ich erst dann in den Parser ein, wenn die Zuweisungen funktionieren und dafür auch Code erzeugt wird. Und vielleicht kommen noch andere Dinge dazu, die ich vorher mache. Also kurz gesagt, das saubere Umsetzen der kompletten Grammatik wird nicht heute geschehen ;-)

Der Hack für `xtest_parser.abc` sieht bei mir so aus:

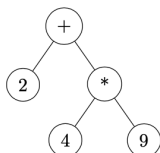
```

1 @ <stdio.h>
2 @ "gen.hdr"
3 @ "lexer.hdr"
4 @ "parser.hdr"
5
6 fn main()
7 {
8     getToken();
9     // parse program (sequence of statements)
10    while (true) {
11        // parse statement
12        local expr: -> Expr = parseExpr();
13        if (!expr) {
14            break;
15        }
16        if (token.kind != SEMICOLON) {
17            printf("%u.%u: ';' expected\n", token.pos.row, token.pos.col);
18            break;
19        }
20        getToken();
21        printExprLatexTree(expr);
22    }
23    if (token.kind != EOI) {
24        printf("syntax error\n");
25    }
26 }

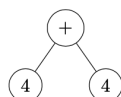
```

Durch die Kommentare ist bereits angedeutet, dass hier in `main` eigentlich zwei Parse-Funktionen direkt implementiert werden. In `parser.hdr` und `parser.abc` wurde noch nichts verändert. Mit dem neuen Testprogramm kann man jetzt aber für ein Programm die Sequenz von Expression-Statements mit LaTeX visualisieren. Zum Beispiel habe ich das Bild

1.



2.



erzeugt, indem ich zunächst

```
MCL:s19 lehn$ (cat tree header.tex; ./xtest parser; cat tree footer.tex) > tree.tex
```

ausgeführt habe und dann die Zeilen

```
2 + 4 * 9;
4 + 4;
```

gefolgt von `Control-D` eingetippt habe. Die erzeugte Datei `tree.tex` kann man dann wieder mit `lualatex` übersetzen.

Aufgabe: Nachmachen!

Schritt 5 (Etwas Shell-Magic)

Auf Dauer wird es immer mühsamer, jedes Mal ein Programm einzutippen, um unseren Parser oder Compiler zu testen. Schreibt in eine Datei `foo` die Zeilen:

```
2 + 4 * 9;  
4 + 4;
```

Und führt dann folgendes Kommando aus:

```
cat foo | (cat tree header.tex; ./xtest parser; cat tree footer.tex) > tree.tex
```

Mit dem ersten `cat`-Kommando wird der Inhalt von `foo` ausgegeben. Wegen dem darauffolgenden sogenannten Pipe-Symbol `|` wird der Inhalt aber nicht im Terminal ausgegeben, sondern an das nächste Kommando weitergereicht. Und zwar so, dass das zweite Kommando meint, es bekommt die Eingabe von der Tastatur.

Aufgabe

Ändert für die weiteren Schritte den Inhalt von `foo` zu:

```
x + 2 + 4 * 9;  
y + x;
```

Schritt 6

Einen Identifier kann man analog zu einem Decimal-Literal in die Grammatik einbauen:

```
prog = { statement }  
statement = expr ";"  
expr = term { ( "+" | "-" ) term }  
term = factor { ( "*" | "/" ) factor }  
factor = decimal-literal  
        | identifier  
        | "(" expr ")"
```

Man muss also nur in `parseFactor` eine weitere Fallunterscheidung einbauen.

Aufgabe

Macht das und testet die Änderung. Wenn alles klappt, ändert `foo` für den nächsten Schritt zu:

```
x = 2 + 4 * 9;  
y = x;
```

Schritt 7

Im ersten Entwurf (ja, es wird einen zweiten geben) für eine Grammatik mit Zuweisungen habe ich die bisherige Grammatik abgeändert und ergänzt zu:

```
prog = { statement }  
statement = expr ";"  
expr = additive { "=" additive }  
additive = term { ( "+" | "-" ) term }  
term = factor { ( "*" | "/" ) factor }  
factor = decimal-literal | identifier | "(" expr ")"
```

Die Zuweisung wird also analog zu den anderen binären Operatoren behandelt. Zur Erinnerung, je weiter unten eine Produktionsregel auftaucht, umso stärker bindet der Operator. Zum Beispiel wird "Punkt vor Strich" unterstützt, da die Regel mit `*` und `/` nach der Regel mit `+` und `-` kommt.

Im Prinzip habe ich die Produktionsregel für `=` kopiert:

- Bei der Kopie wurde die linke Seite (also `expr`) nur in **additive** umbenannt.
- In der Zeile darüber wurde nur die rechte Seite verändert, die Produktionsregel greift auf **additive** zurück und es kommt `=` vor (statt `+` oder `-`).

Analog (also mit Kopieren und Ändern) kann man bei der Implementierung der Parse-Funktionen vorgehen. Falls beim Test ein Assertion-Failure wie zum Beispiel

`parser.abc:41: Assertion '0 && "internal error: exprKindFromToken"' failed.` vorkommt, ist das kein großes Problem. In der Fehlermeldung steht ja, an welcher Stelle man noch etwas anpassen muss.

Schritt 8

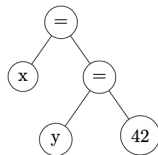
Ein Problem haben wir noch. Testet folgende Expression-Statements:

`x = (y = 42);`

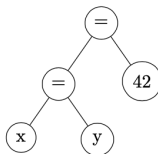
`x = y = 42;`

So wie wir das aus C oder ABC kennen, sollte beides die gleiche Bedeutung haben. Erst soll `42` der Variable `y` zugewiesen werden, dann sollte der Wert von `y` der Variable `x` zugewiesen werden. Wenn wir aber die Bildchen anschauen, die unser Parser generiert, sehen wir folgendes:

1.



2.



Die Bilder kann man so interpretieren: Bei einem Knoten mit `=` wird der rechte Child-Node dem linken Child-Node zugewiesen. Der erste Expression-Tree ist der, den wir in C und ABC in beiden Fällen erhalten würden. Damit auch unser Compiler das liefert, muss die Grammatik angepasst werden, damit Zuweisungen von rechts nach links ausgewertet werden (man sagt, dass der Zuweisungsoperator "rechts assoziativ" ist) und nicht wie bei den anderen binären Operatoren von links nach rechts (man nennt diese "links assoziativ"). Hier die korrigierte Grammatik:

```

prog = { statement }
statement = expr ";"
expr = additive { "=" expr }
additive = term { ( "+" | "-" ) term }
term = factor { ( "*" | "/" ) factor }
factor = decimal-literal | identifier | "(" expr ")"
  
```

Geändert wurde hier nur, dass bei der Produktionsregel für `expr` nach dem `=` die Regel `expr` rekursiv vorkommt. Und genau so, also mit einem rekursiven Funktionsaufruf, wird dann auch die zugehörige Parse-Funktion implementiert.

Aufgabe: Passt den Parser an und testet ihn.